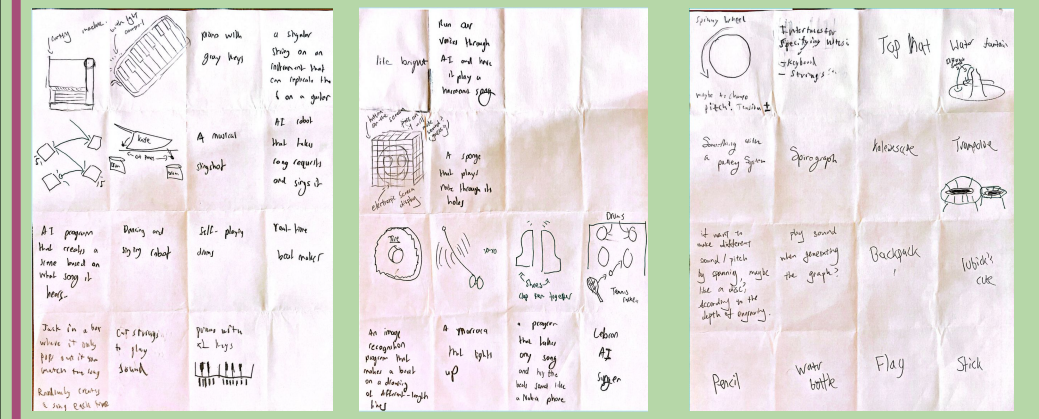
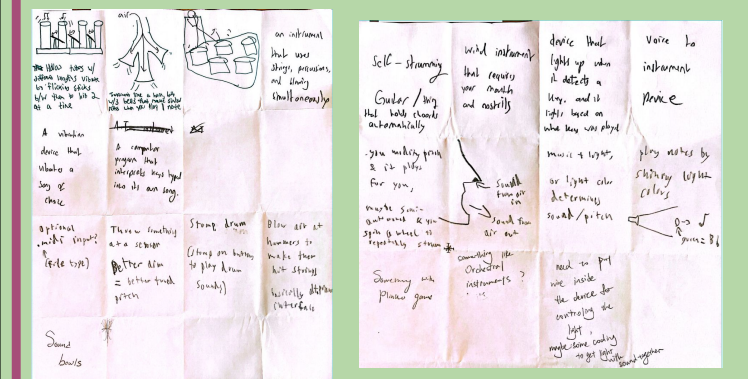
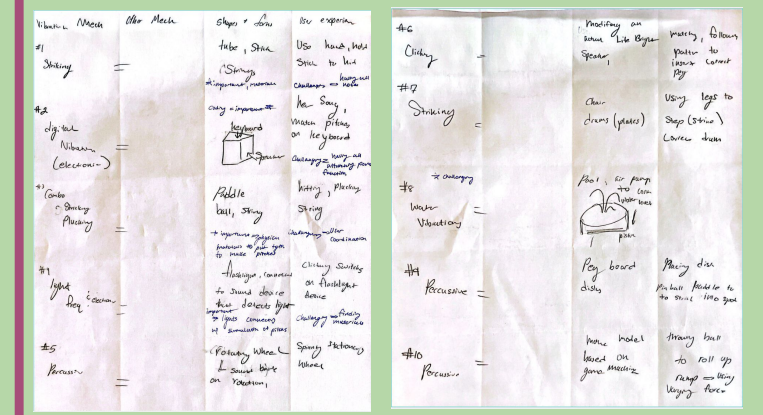
Brainstorming Phase





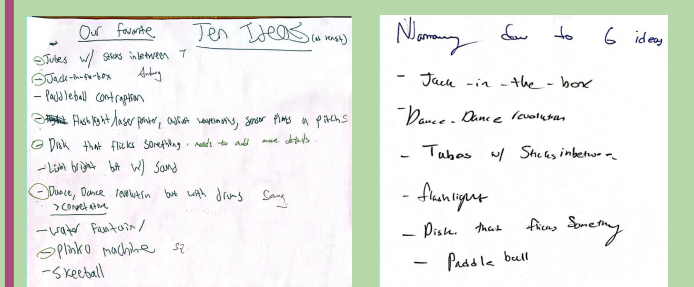
Description
- Rapid-fire concepts
- Roughly 10-15 seconds per idea
- Entire team worked together
- Passed around papers after each row to build off others' ideas

Learned:
- Brainstorming requires practice
- Some of us are better rapid fire thinkers
- Others are more detail-oriented
- Detail-oriented people may struggle with this phase



Description
- Relating vibration mechanics to:
- Shapes and Form
- User experience

Learned
- How to identify important considerations in design
- How to identify challenges with the designs



Our favorite Ten Ideas (at least)

- Tubes w/ sticks inbetween
- Jack-in-the-box Aubry
- Paddleball contraption
- Flashlight/laser pointer, overwise movements, sensor plays in pitches
- Disk that flicks something. needs to add more details
- Lion bright bot w/ Sang
- Dance, Dance revolution but with drums Sang
- Water Fountain/
- Plinko machine
- Skeeball

Narrowing down to 6 ideas

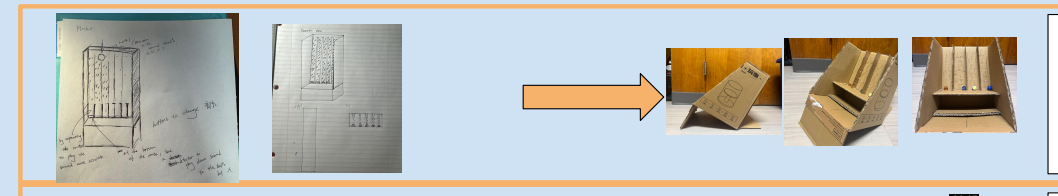
- Jack-in-the-box
- Dance-Dance revolution
- Tubes w/ Sticks inbetween
- flashlight
- Disk that flicks something
- Paddle ball

Description
- Narrowing ideas down
- Picked top 10
- Then picked top 6

Learned:
- Importance of "spamming" ideas
- Usefulness of building brainstorming idea off each other

Individual Design Concepts

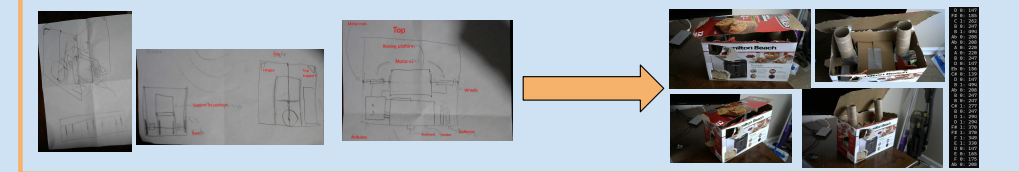
Sophie



Description
- Built the Plinko with a cardboard box
- Added nails in varied patterns so marbles create sounds
- Tilted and reinforced for stability

Learned:
- How to organize the position for the nails so the marbles can roll smoothly
- There is still possibility that the different material marbles will get stuck when drop it from the top

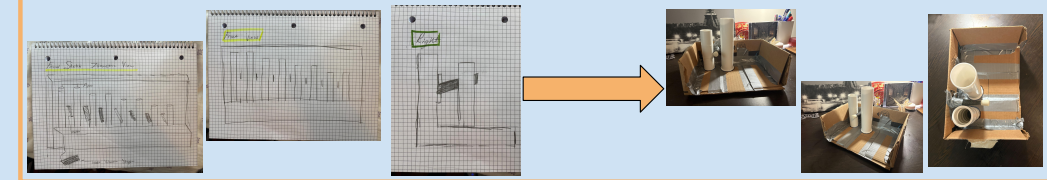
Aubrey



Description
- Raising and lowering the platform also affects the lid
- Program generates random sequences of notes
- Note generation is weighted to prefer natural notes and closer notes, hopefully producing nicer melodies

Learned:
- A spinning wheel and axle will not work well to raise a platform
- The required program is simple and should be fairly straightforward to replicate on an arduino

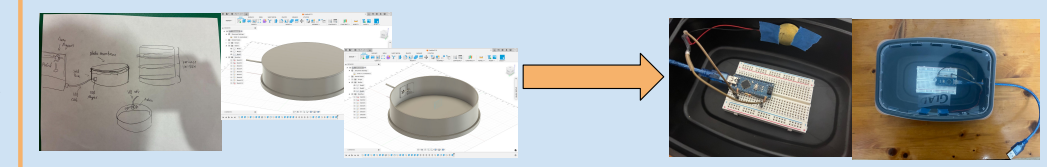
Sophia



Description
- Built base then attached pipes and doorstop with superglue and/or duct tape
- Flick doorstop in either right or left direction to strike respective pipe
  - Different pipes create different sound

Learned:
- Upon drying of the final product, the same sounds were not produced
  - Not as defined compared to when setting up prototype

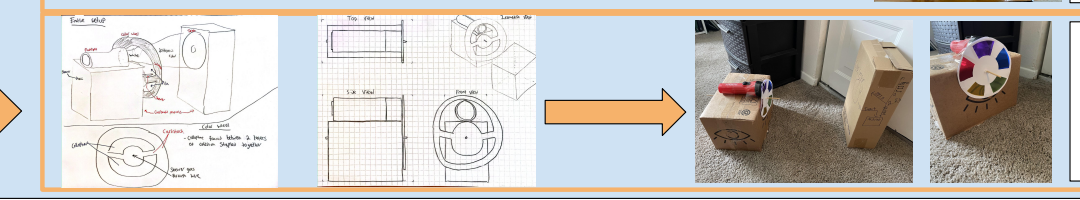
Sang



Description
- Strike Drum to allow Piezo to receive vibrations
- Arduino reads input from Piezo, and outputs to computer
- Has 1 second delay between each strikes and has a vibration strength threshold

Learned:
- How to utilize a breadboard and its row-based circuitry.
- Familiarized myself with the arduino IDE
- Learned how to connect wires between receiver(piezo) and jumper wires.

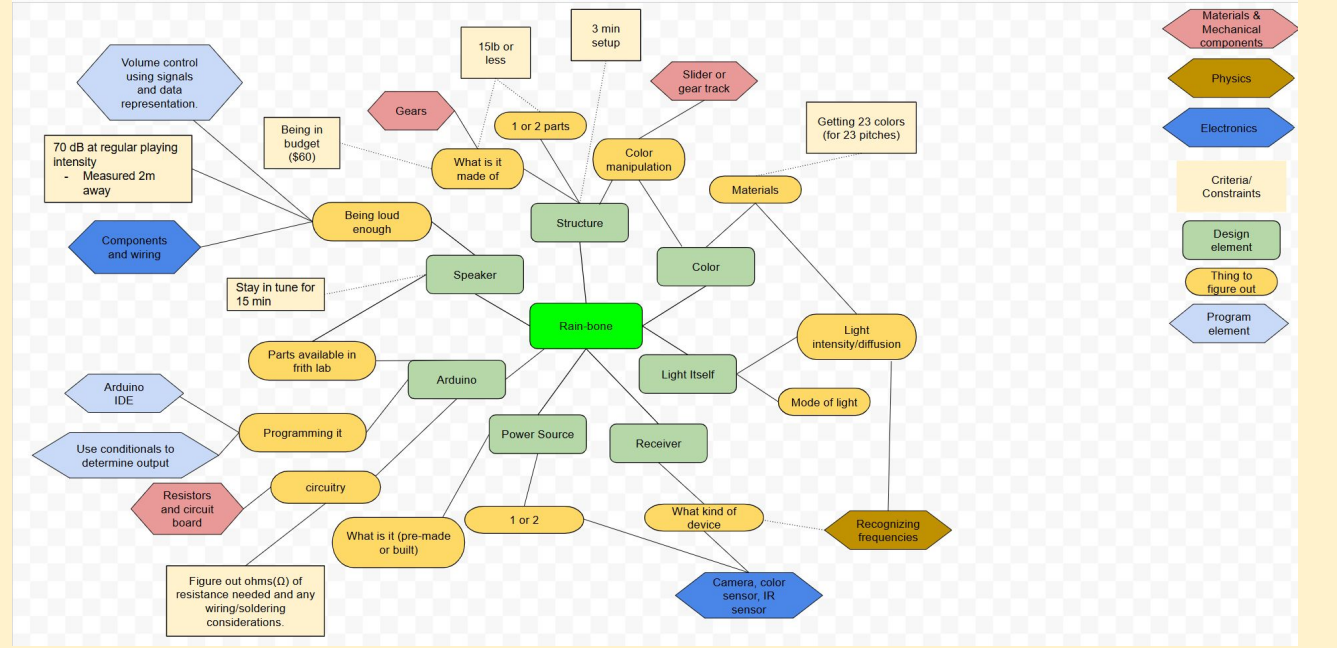
Tanner



Description
- Light shines from LEDs
- Change color with a wheel
- Hits a target
- Receiver plays notes fixed to lightwave frequencies

Learned:
- Cellophane changed color well
- LED able to reach receiver well
- Some colors not as strong as others
- Too many layers of cellophane diffuses light too much

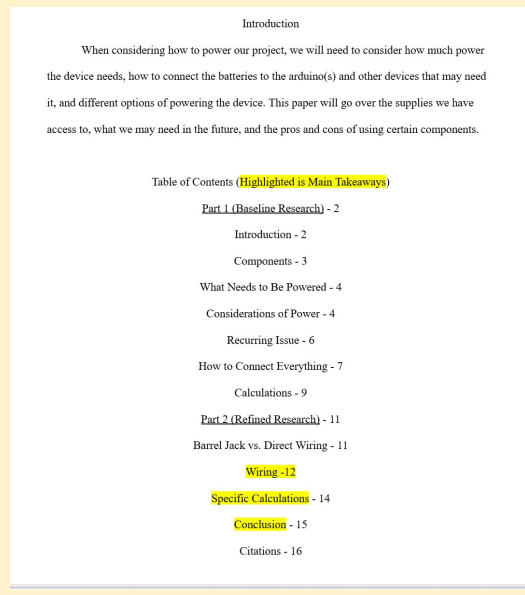
First Complete Prototype



Description
- Scope map for our entire plan
- Includes breakdown of individual design elements
- Includes considerations for each element

Learned:
- Insight into what needed to be done for ext prototype
- Laid out factors we have to consider moving forward

Initial Research

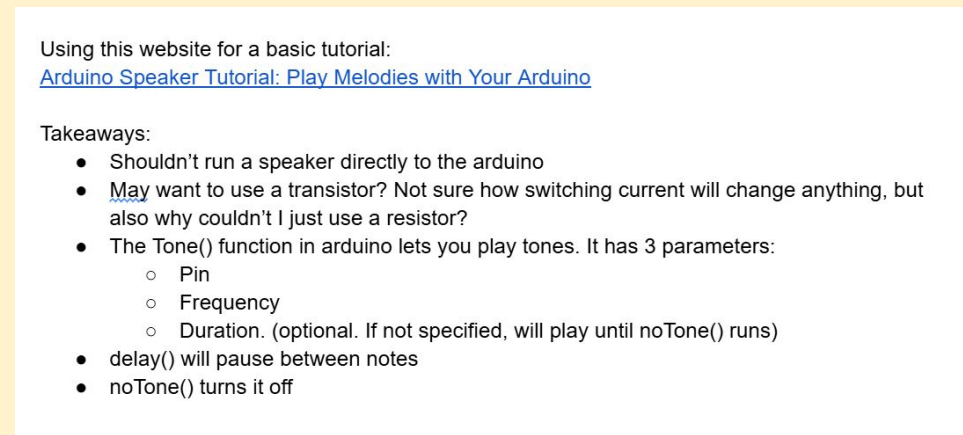


Introduction

When considering how to power our project, we will need to consider how much power the device needs, how to connect the batteries to the arduino(s) and other devices that may need it, and different options of powering the device. This paper will go over the supplies we have access to, what we may need in the future, and the pros and cons of using certain components.

Table of Contents (Highlighted is Main Takeaways)

Part 1 (Baseline Research) - 2
Introduction - 2
Components - 3
What Needs to Be Powered - 4
Considerations of Power - 4
Recurring Issue - 6
How to Connect Everything - 7
Calculations - 9
Part 2 (Refined Research) - 11
Barrel Jack vs. Direct Wiring - 11
Wiring -12
Specific Calculations - 14
Conclusion - 15
Citations - 16

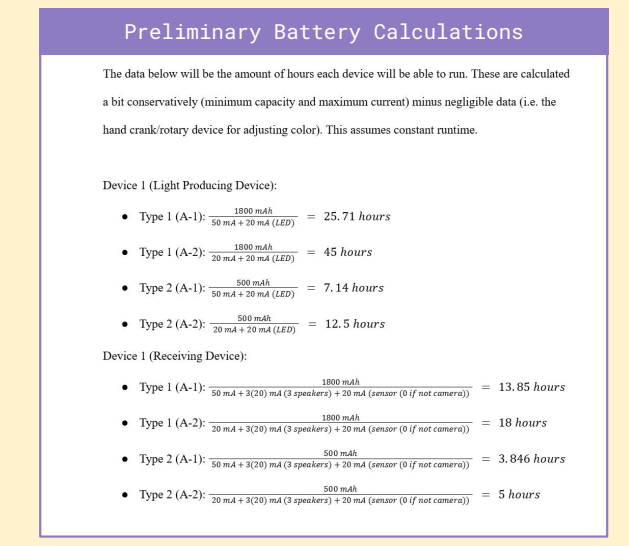


Using this website for a basic tutorial:
Arduino Speaker Tutorial: Play Melodies with Your Arduino

Takeaways:
- Shouldn't run a speaker directly to the arduino
- May want to use a transistor? Not sure how switching current will change anything, but also why couldn't I just use a resistor?
- The Tone() function in arduino lets you play tones. It has 3 parameters:
  - Pin
  - Frequency
  - Duration. (optional. If not specified, will play until noTone() runs)
- delay() will pause between notes
- noTone() turns it off

Description
- Component research: battery, color sensors, and speakers
- Includes individual breakout research done on specific components before our first build

Learned:
- Arduino and many components want 5v, need to scale to that
- Our best bet would be one or more 9v batteries in series
- Speakers shouldn't be plugged directly to arduino
- How to wire all the components, including the various pins of the TCS3200 color sensor
- How to wire an RGB LED

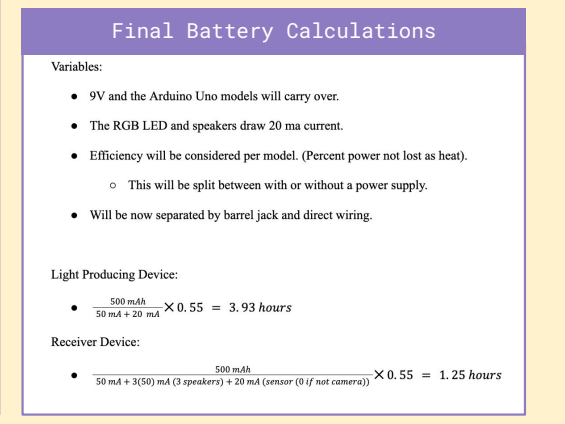


Preliminary Battery Calculations

The data below will be the amount of hours each device will be able to run. These are calculated a bit conservatively (minimum capacity and maximum current) minus negligible data (i.e. the hand crank/rotary device for adjusting color). This assumes constant runtime.

Device 1 (Light Producing Device):
- Type 1 (A-1): $\frac{1800\ mAh}{50\ mA + 20\ mA\ (LED)} = 25.71\ hours$
- Type 1 (A-2): $\frac{1800\ mAh}{20\ mA + 20\ mA\ (LED)} = 45\ hours$
- Type 2 (A-1): $\frac{500\ mAh}{50\ mA + 20\ mA\ (LED)} = 7.14\ hours$
- Type 2 (A-2): $\frac{500\ mAh}{20\ mA + 20\ mA\ (LED)} = 12.5\ hours$

Device 1 (Receiving Device):
- Type 1 (A-1): $\frac{1800\ mAh}{50\ mA + 3(20)\ mA\ (3\ speakers) + 20\ mA\ (sensor\ (0\ if\ not\ camera))} = 13.85\ hours$
- Type 1 (A-2): $\frac{1800\ mAh}{20\ mA + 3(20)\ mA\ (3\ speakers) + 20\ mA\ (sensor\ (0\ if\ not\ camera))} = 18\ hours$
- Type 2 (A-1): $\frac{500\ mAh}{50\ mA + 3(20)\ mA\ (3\ speakers) + 20\ mA\ (sensor\ (0\ if\ not\ camera))} = 3.846\ hours$
- Type 2 (A-2): $\frac{500\ mAh}{20\ mA + 3(20)\ mA\ (3\ speakers) + 20\ mA\ (sensor\ (0\ if\ not\ camera))} = 5\ hours$



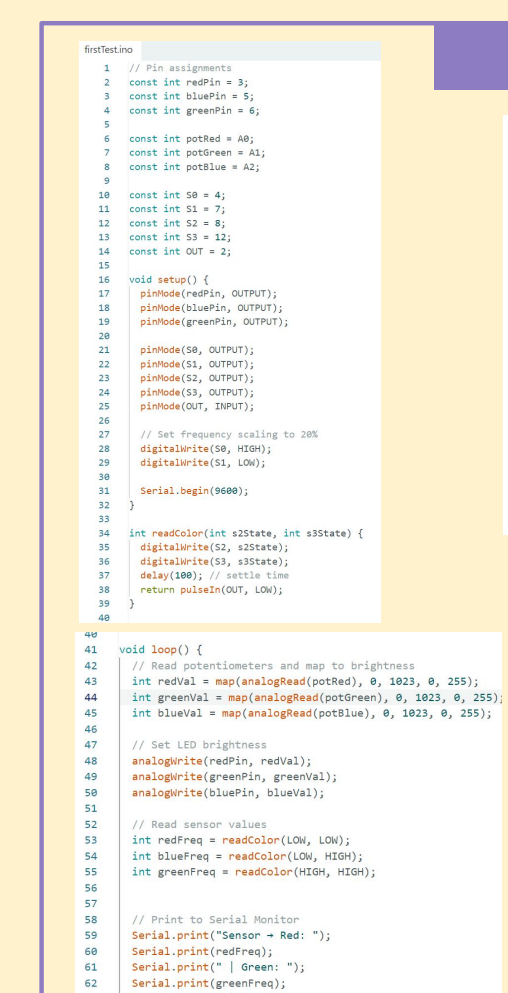
Final Battery Calculations

Variables:
- 9V and the Arduino Uno models will carry over.
- The RGB LED and speakers draw 20 ma current.
- Efficiency will be considered per model. (Percent power not lost as heat).
  - This will be split between with or without a power supply.
- Will be now separated by barrel jack and direct wiring.

Light Producing Device:
- $\frac{500\ mAh}{50\ mA + 20\ mA} \times 0.55 = 3.93\ hours$

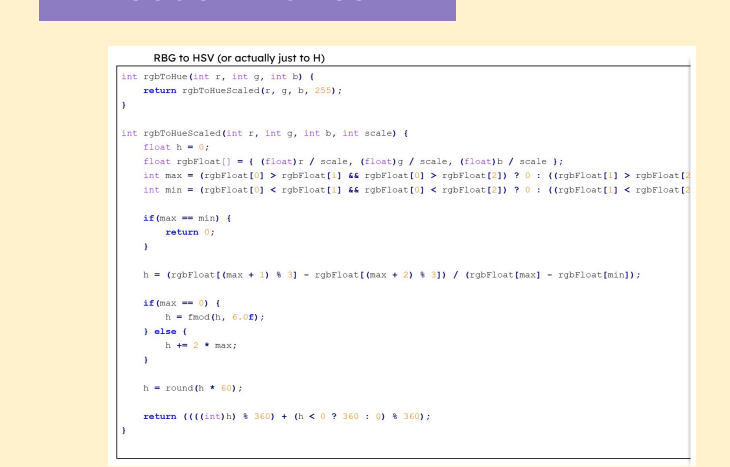
Receiver Device:
- $\frac{500\ mAh}{50\ mA + 3(50)\ mA\ (3\ speakers) + 20\ mA\ (sensor\ (0\ if\ not\ camera))} \times 0.55 = 1.25\ hours$

Code Drafts



firstTest.ino

```
// Pin assignments
const int redPin = 3;
const int bluePin = 5;
const int greenPin = 6;

const int potRed = A0;
const int potGreen = A1;
const int potBlue = A2;

const int S0 = 4;
const int S1 = 7;
const int S2 = 8;
const int S3 = 12;
const int OUT = 2;

void setup() {
  pinMode(redPin, OUTPUT);
  pinMode(bluePin, OUTPUT);
  pinMode(greenPin, OUTPUT);

  pinMode(S0, OUTPUT);
  pinMode(S1, OUTPUT);
  pinMode(S2, OUTPUT);
  pinMode(S3, OUTPUT);
  pinMode(OUT, INPUT);

  // Set frequency scaling to 20%
  digitalWrite(S0, HIGH);
  digitalWrite(S1, LOW);

  Serial.begin(9600);
}

int readColor(int s2State, int s3State) {
  digitalWrite(S2, s2State);
  digitalWrite(S3, s3State);
  delay(100); // settle time
  return pulseIn(OUT, LOW);
}

void loop() {
  // Read potentiometers and map to brightness
  int redVal = map(analogRead(potRed), 0, 1023, 0, 255);
  int greenVal = map(analogRead(potGreen), 0, 1023, 0, 255);
  int blueVal = map(analogRead(potBlue), 0, 1023, 0, 255);

  // Set LED brightness
  analogWrite(redPin, redVal);
  analogWrite(greenPin, greenVal);
  analogWrite(bluePin, blueVal);

  // Read sensor values
  int redFreq = readColor(LOW, LOW);
  int blueFreq = readColor(LOW, HIGH);
  int greenFreq = readColor(HIGH, HIGH);


  // Print to Serial Monitor
  Serial.print("Sensor → Red: ");
  Serial.print(redFreq);
  Serial.print(" | Green: ");
  Serial.print(greenFreq);
  Serial.print(" | Blue: ");
  Serial.println(blueFreq);

  delay(3000); // wait 3 seconds before next reading
}
```



RBG to HSV (or actually just to H)

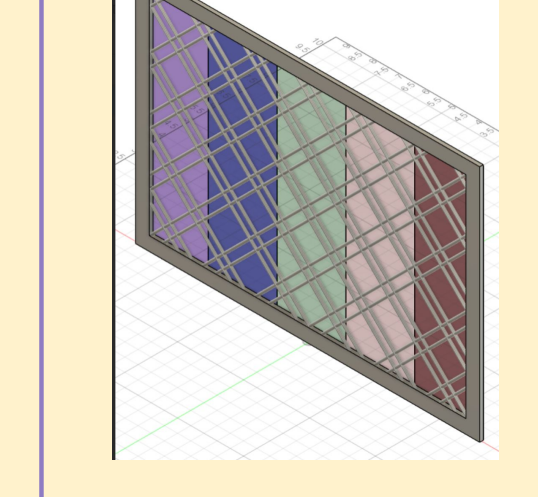
```
int rgbToHue(int r, int g, int b) {
    return rgbToHueScaled(r, g, b, 255);
}

int rgbToHueScaled(int r, int g, int b, int scale) {
    float h = 0;
    float rgbFloat[] = { (float)r / scale, (float)g / scale, (float)b / scale };
    int max = (rgbFloat[0] > rgbFloat[1] && rgbFloat[0] > rgbFloat[2]) ? 0 : ((rgbFloat[1] > rgbFloat[2
    int min = (rgbFloat[0] < rgbFloat[1] && rgbFloat[0] < rgbFloat[2]) ? 0 : ((rgbFloat[1] < rgbFloat[2

    if(max == min) {
        return 0;
    }

    h = (rgbFloat[(max + 1) % 3] - rgbFloat[(max + 2) % 3]) / (rgbFloat[max] - rgbFloat[min]);

    if(max == 0) {
        h = fmod(h, 6.0f);
    } else {
        h += 2 * max;
    }

    h = round(h * 60);

    return ((((int)h) % 360) + (h < 0 ? 360 : 0) % 360);
}
```

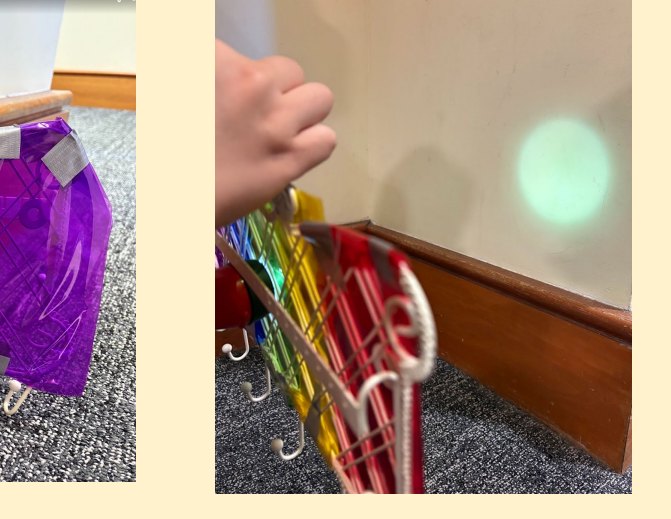
Description
- Includes a program to test LED, Color sensor, and potentiometers
- Draft code for converting RGB values into 1 hue value

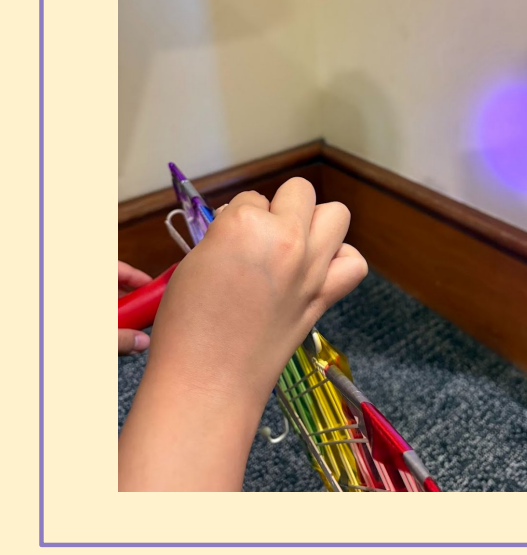
Learned:
- How to program LEDs, color sensor, speakers, potentiometers, and the arduino itself
- Possible ways we can manipulate color with one variable instead of 3

First Complete Prototype

Preliminary Input Medium



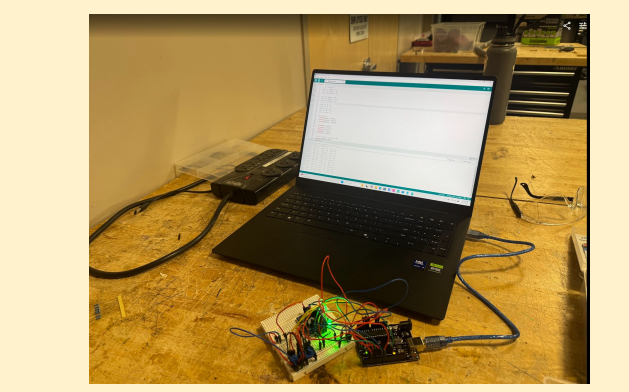




Description
- Designing and building temporary input system
- Support lattice with cellophane color filters

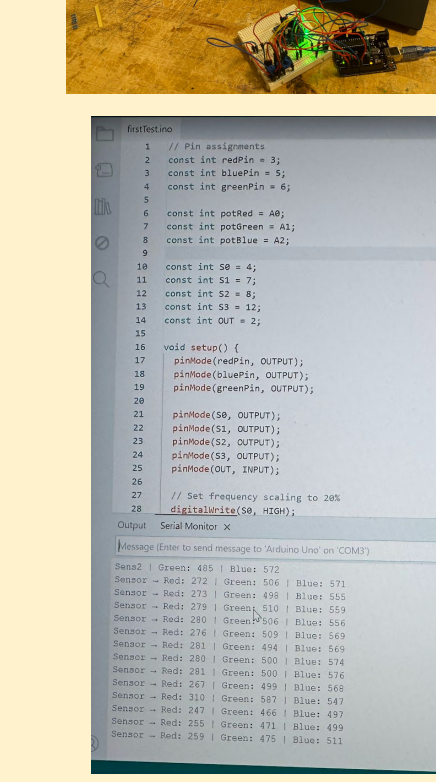
Learned:
- Easier to control color on a stationary lattice than with a handless wheel
- White LED light through cellophane does not produce focused or bright enough light for color sensor, even at extremely close range
- Will likely abandon this method of color changing

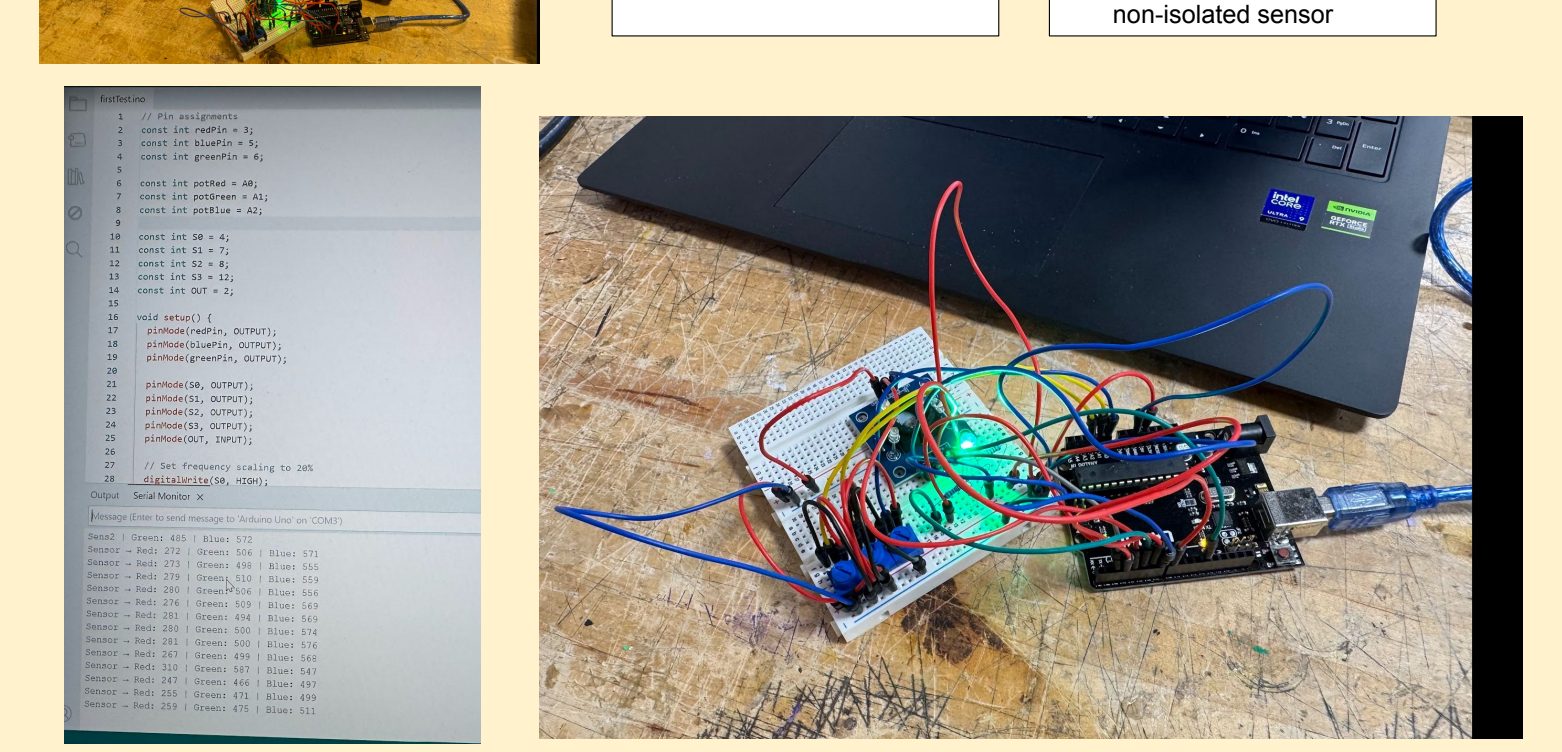
Testing



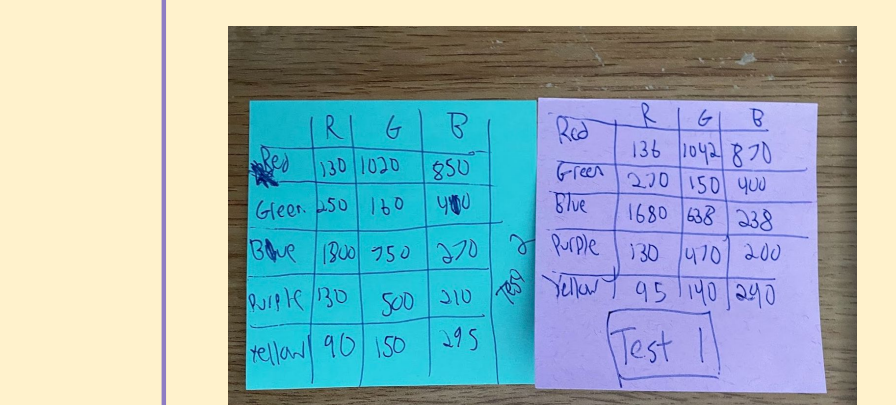
Description
- First component test
- Created arduino script
- Testing LED, potentiometer, and Color sensor wiring and cooperation

Learned:
- How to wire the components
- Color sensor produces unreliable results in current setup
- Possible causes include ambient light, unfocused light source, and non-isolated sensor



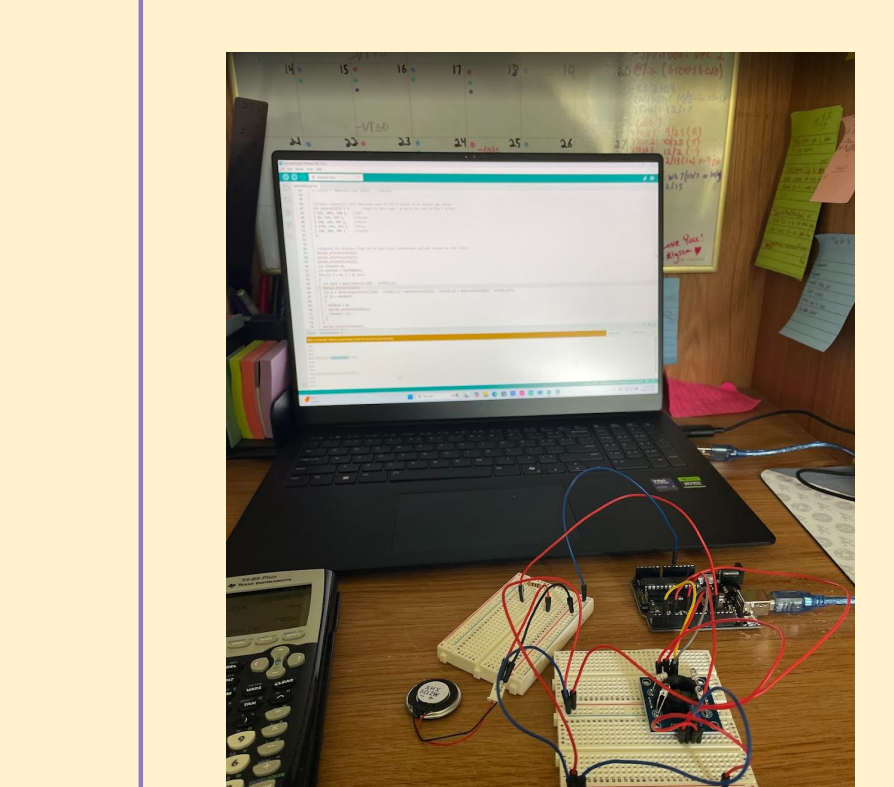


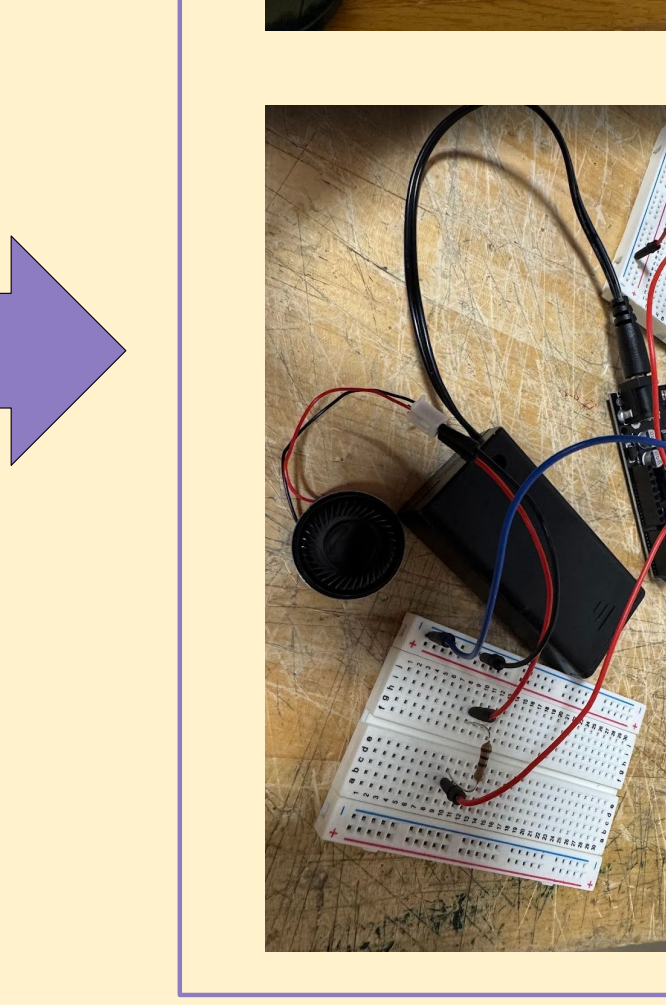
First Complete Prototype

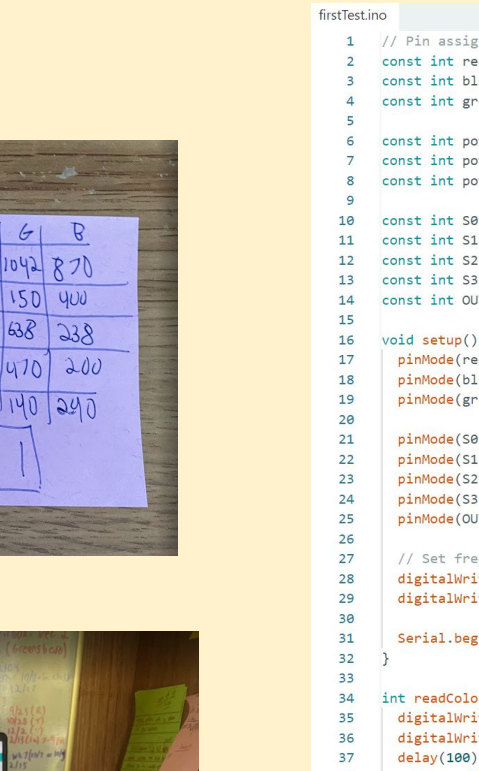


	R	G	B
Red	136	1042	870
Green	220	150	400
Blue	1680	638	238
Purple	130	470	200
Yellow	95	140	290

Test 1

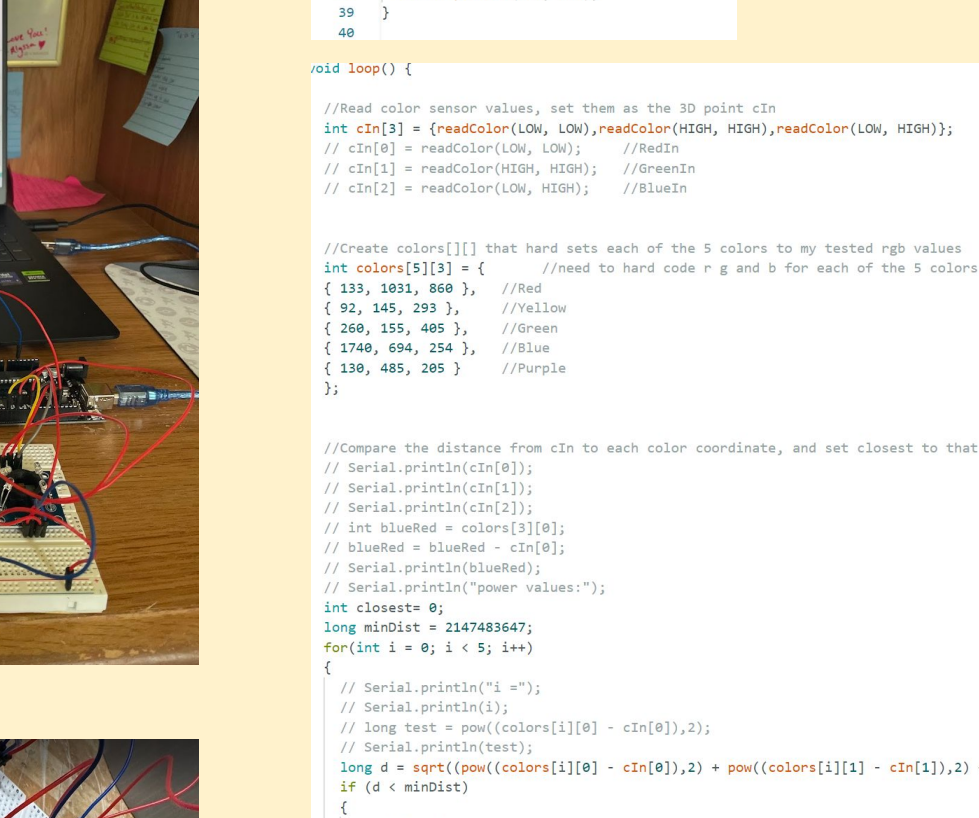




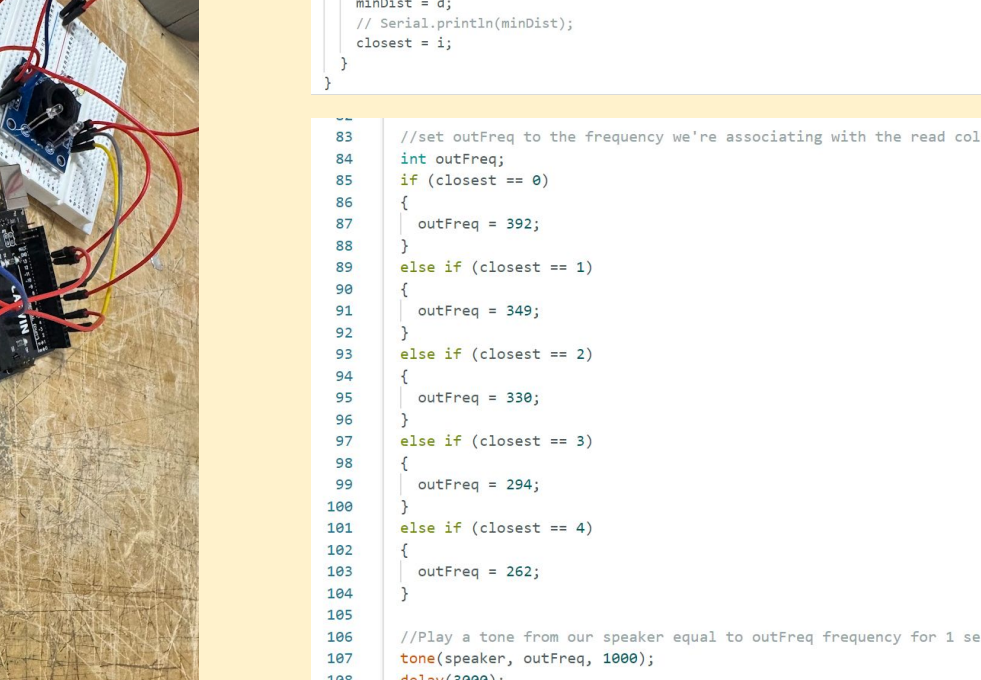


firstTest.ino

```
// Pin assignments
const int redPin = 3;
const int bluePin = 5;
const int greenPin = 6;

const int potRed = A0;
const int potGreen = A1;
const int potBlue = A2;

const int S0 = 4;
const int S1 = 7;
const int S2 = 8;
const int S3 = 12;
const int OUT = 2;

void setup() {
  pinMode(redPin, OUTPUT);
  pinMode(bluePin, OUTPUT);
  pinMode(greenPin, OUTPUT);

  pinMode(S0, OUTPUT);
  pinMode(S1, OUTPUT);
  pinMode(S2, OUTPUT);
  pinMode(S3, OUTPUT);
  pinMode(OUT, INPUT);

  // Set frequency scaling to 20%
  digitalWrite(S0, HIGH);
  digitalWrite(S1, LOW);

  Serial.begin(9600);
}

int readColor(int s2State, int s3State) {
  digitalWrite(S2, s2State);
  digitalWrite(S3, s3State);
  delay(100); // settle time
  return pulseIn(OUT, LOW);
}
```



```
void loop() {

 //Read color sensor values, set them as the 3D point cIn
 int cIn[3] = {readColor(LOW, LOW),readColor(HIGH, HIGH),readColor(LOW, HIGH)};
 // cIn[0] = readColor(LOW, LOW);     //RedIn
 // cIn[1] = readColor(HIGH, HIGH);   //GreenIn
 // cIn[2] = readColor(LOW, HIGH);    //BlueIn


 //Create colors[][] that hard sets each of the 5 colors to my tested rgb values
 int colors[5][3] = {       //need to hard code r g and b for each of the 5 colors
 { 133, 1031, 860 },   //Red
 { 92, 145, 293 },     //Yellow
 { 260, 155, 405 },    //Green
 { 1740, 694, 254 },   //Blue
 { 130, 485, 205 }     //Purple
 };


 //Compare the distance from cIn to each color coordinate, and set closest to that color
 // Serial.println(cIn[0]);
 // Serial.println(cIn[1]);
 // Serial.println(cIn[2]);
 // int blueRed = colors[3][0];
 // blueRed = blueRed - cIn[0];
 // Serial.println(blueRed);
 // Serial.println("power values:");
 int closest= 0;
 long minDist = 2147483647;
 for(int i = 0; i < 5; i++)
 {
  // Serial.println("i =");
  // Serial.println(i);
  // long test = pow((colors[i][0] - cIn[0]),2);
  // Serial.println(test);
  long d = sqrt((pow((colors[i][0] - cIn[0]),2) + pow((colors[i][1] - cIn[1]),2) + pow((colors[i][2] - cIn[2]),2)));
  if (d < minDist)
  {
   minDist = d;
   // Serial.println(minDist);
   closest = i;
  }
 }
```



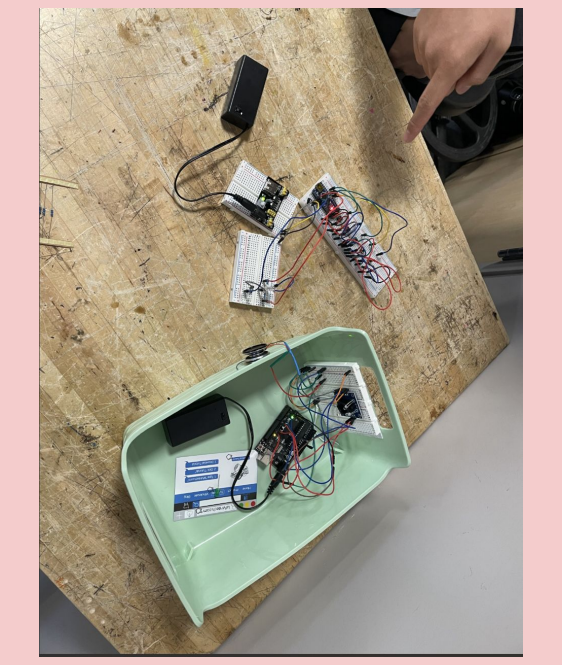
```
  //set outFreq to the frequency we're associating with the read color
  int outFreq;
  if (closest == 0)
  {
   outFreq = 392;
  }
  else if (closest == 1)
  {
   outFreq = 349;
  }
  else if (closest == 2)
  {
   outFreq = 330;
  }
  else if (closest == 3)
  {
   outFreq = 294;
  }
  else if (closest == 4)
  {
   outFreq = 262;
  }

  //Play a tone from our speaker equal to outFreq frequency for 1 second, then wait for a 3 seconds
  tone(speaker, outFreq, 1000);
  delay(3000);
```

Description
- Final version of this prototyping phase
- Includes a color sensor, speaker, and battery pack wired to an arduino uno with our final program uploaded to it
- Can read colors from an up-close phone image to produce preset tones from the speaker
- Runs on its own battery source

Learned
- Color sensor is very sensitive, and will require precise and smart programming and physical construction to produce desired results
- Input system has to produce a very bright and focused light
- May need to isolate the color sensor

Moving Forward:
- Need to move to powerful RGB LEDs as light source
- Need to engineer and research solution to map rgb values to a constant changing light wave frequency(color)
- Explore color sensor isolation in casing construction
- Consider IR light to help isolate only desired inputs as values to be red

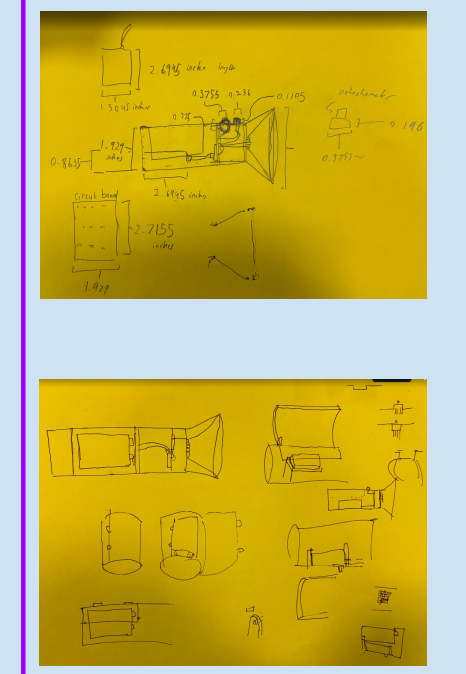
Functional Prototype

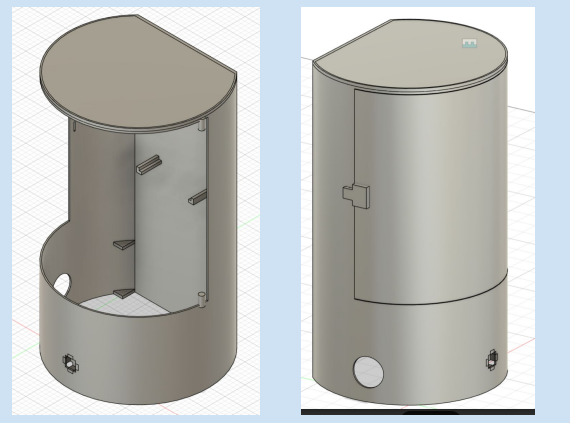


Description
- Next prototype includes a couple RGB LEDs
- LEDs wired to produce 5 preset colors
- Input is 5 buttons to set them
- Attempted to create a body structure

Moving Forward:
- Implement an amplifier to increase volume
- Implement new LED strip to concentrate and increase brightness of light
- Create an analog input system to adjust between a larger range of frequencies
- Create an effective body for both parts of design
- Implement an isolation chamber to help isolate light going to color sensor

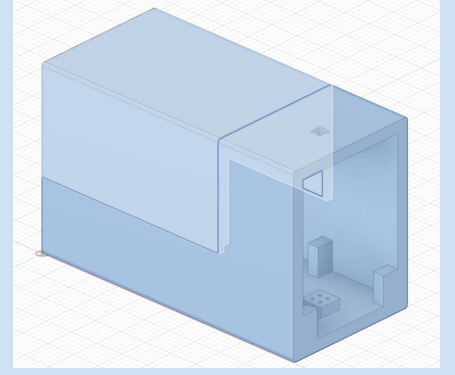
Validated Prototype

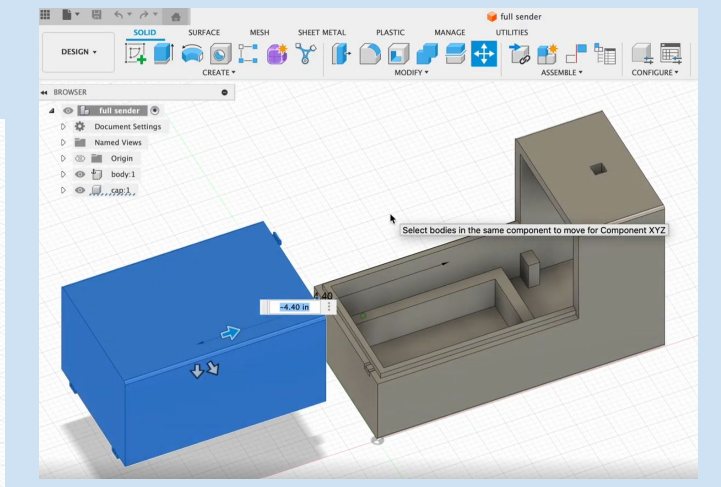


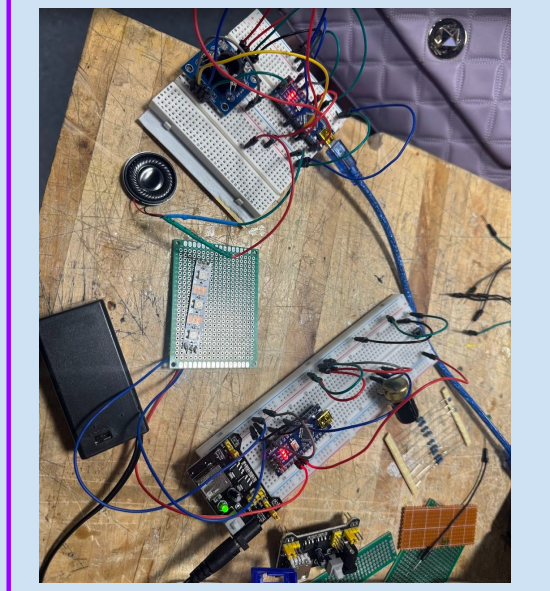


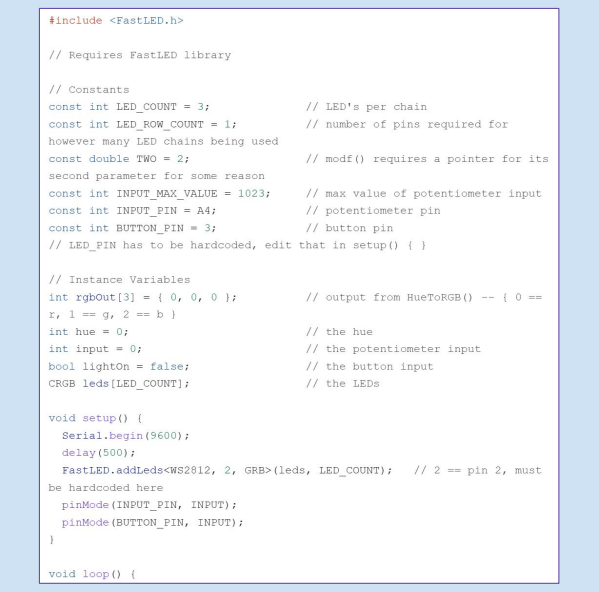
Description:
- Sketches for sender piece designs
- 2 different ideas for sender in CAD
  - One has a hinge door
  - The other has a slider door

What we Learned
- The slider design would likely be a better fit
- Allows for more space to install the wiring and change battery
- Walls have to be a certain thickness for 3D printer to work

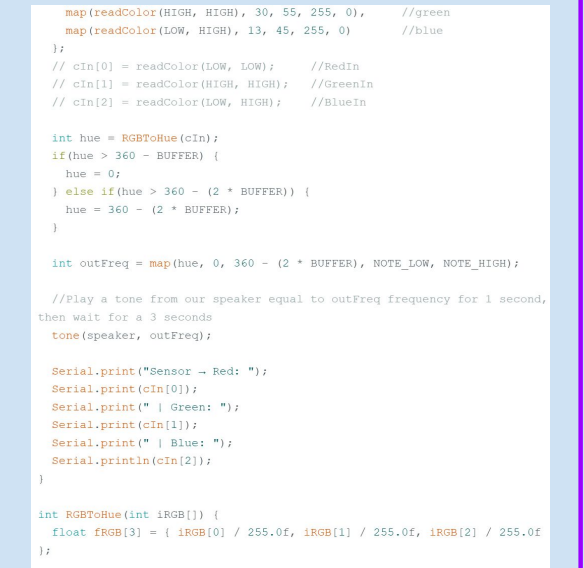








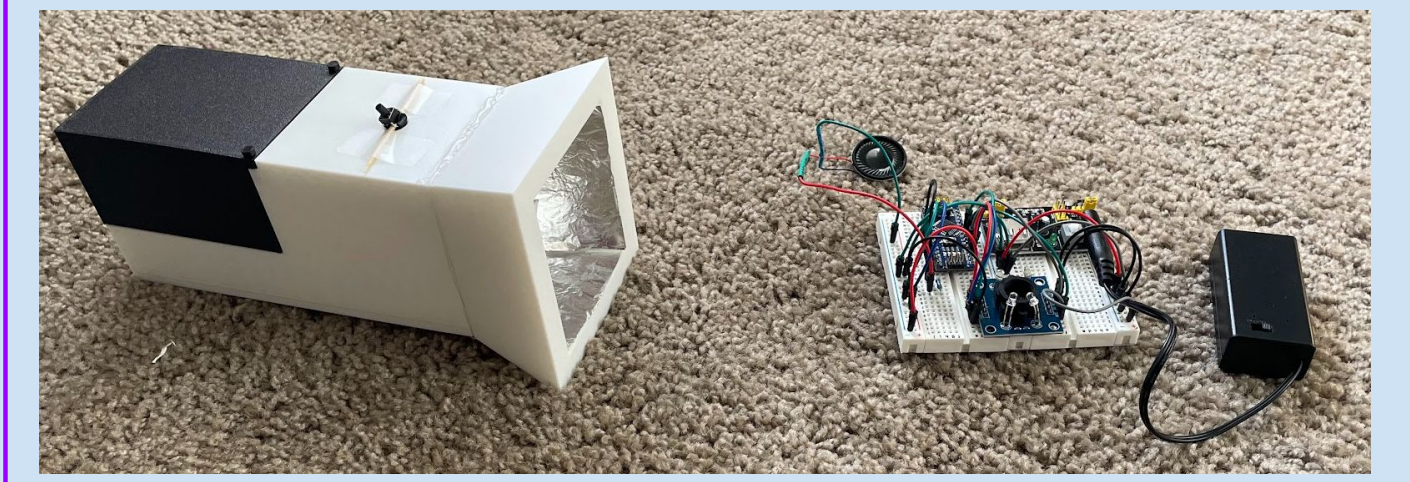
```
#include <FastLED.h>

// Requires FastLED library

// Constants
const int LED_COUNT = 3;              // LED's per chain
const int LED_ROW_COUNT = 1;          // number of pins required for
however many LED chains being used
const double TWO = 2;                 // modf() requires a pointer for its
second parameter for some reason
const int INPUT_MAX_VALUE = 1023;     // max value of potentiometer input
const int INPUT_PIN = A4;             // potentiometer pin
const int BUTTON_PIN = 3;             // button pin
// LED_PIN has to be hardcoded, edit that in setup() { }

// Instance Variables
int rgbOut[3] = { 0, 0, 0 };          // output from HueToRGB() -- { 0 ==
r, 1 == g, 2 == b }
int hue = 0;                          // the hue
int input = 0;                        // the potentiometer input
bool lightOn = false;                 // the button input
CRGB leds[LED_COUNT];                 // the LEDs

void setup() {
  Serial.begin(9600);
  delay(500);
  FastLED.addLeds<WS2812, 2, GRB>(leds, LED_COUNT);   // 2 == pin 2, must
be hardcoded here
  pinMode(INPUT_PIN, INPUT);
  pinMode(BUTTON_PIN, INPUT);
}

void loop() {
```

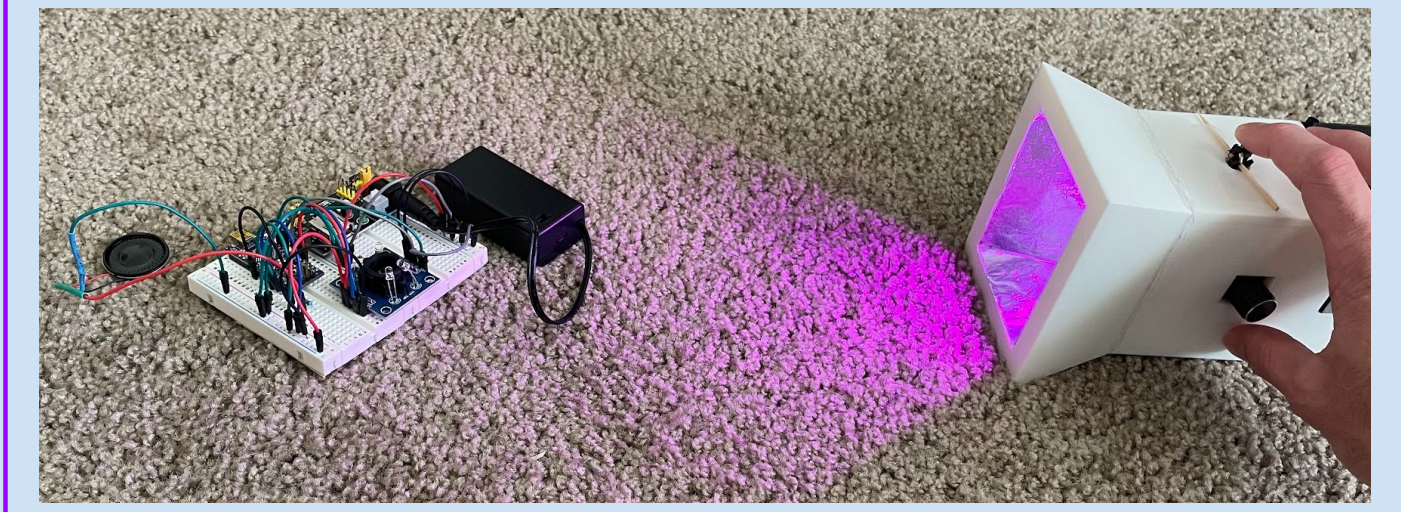


```
    map(readColor(HIGH, HIGH), 30, 55, 255, 0),      //green
    map(readColor(LOW, HIGH), 13, 45, 255, 0)        //blue
  };
  // cIn[0] = readColor(LOW, LOW);     //RedIn
  // cIn[1] = readColor(HIGH, HIGH);   //GreenIn
  // cIn[2] = readColor(LOW, HIGH);    //BlueIn

  int hue = RGBToHue(cIn);
  if(hue > 360 - BUFFER) {
    hue = 0;
  } else if(hue > 360 - (2 * BUFFER)) {
    hue = 360 - (2 * BUFFER);
  }

  int outFreq = map(hue, 0, 360 - (2 * BUFFER), NOTE_LOW, NOTE_HIGH);

  //Play a tone from our speaker equal to outFreq frequency for 1 second,
then wait for a 3 seconds
  tone(speaker, outFreq);

  Serial.print("Sensor → Red: ");
  Serial.print(cIn[0]);
  Serial.print(" | Green: ");
  Serial.print(cIn[1]);
  Serial.print(" | Blue: ");
  Serial.println(cIn[2]);
}

int RGBToHue(int iRGB[]) {
  float fRGB[3] = { iRGB[0] / 255.0f, iRGB[1] / 255.0f, iRGB[2] / 255.0f
};
```

Description:
- New circuitry including soldered led strip
- Features larger potentiometer and single button to control notes
- New sender and receiver code to handle one range of hue values

Learned
- Need to offset hue values to accommodate red color wrapping around hue spectrum
- Need to import fastLED library
- How to wire the LED strip and use one potentiometer for RGB light





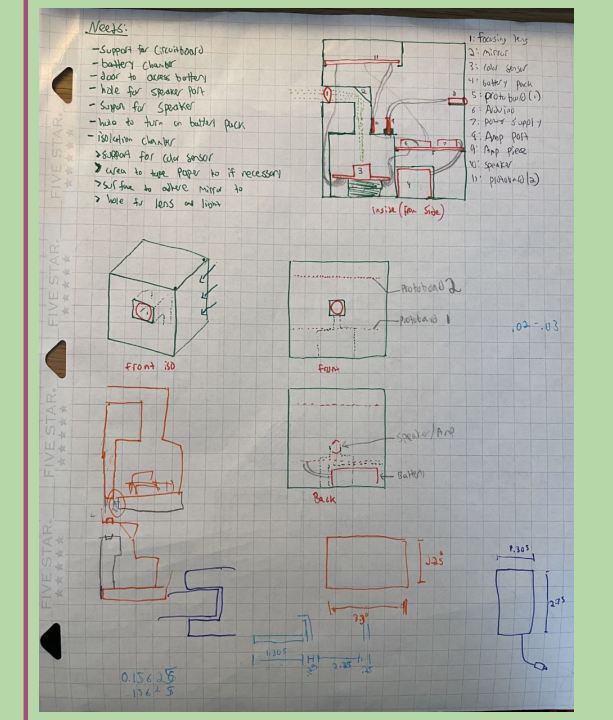
Description
- Validated Prototype with printed sender body
- 3D printed a body that holds sender circuit, battery pack, LEDs, potentiometer, and button
- Has holes for input components, as well as a hole to access the switch for the battery pack
- Pins are put in place to secure the slider design we decided on
- Features a short reflector piece on the end in an attempt to reflect and focus some of the light

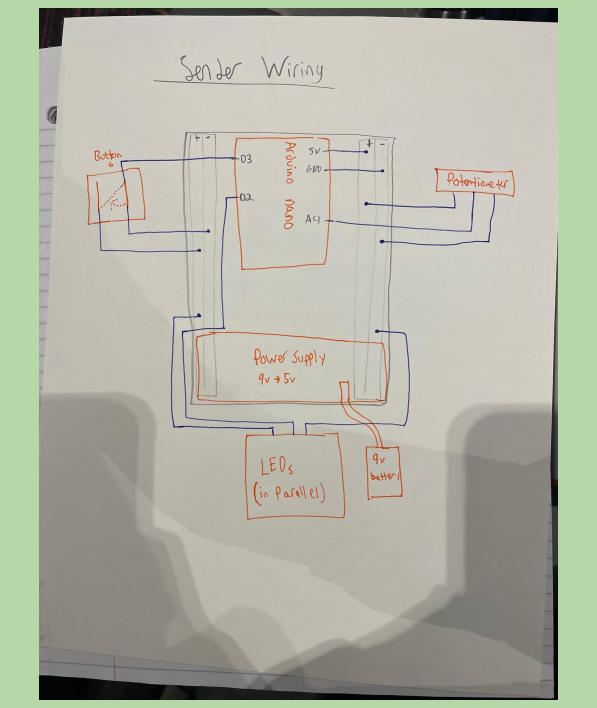
Moving Forward:
- For the final design, we will redo the sender body to be more ergonomic, as well as add something to indicate what color and/or note is being produced
- Need to create a body for the receiver circuit, including an isolation changer for the color sensor, support for multiple protoboard circuits, a large door to access inner mechanisms, and a setup for the speaker and amp pieces

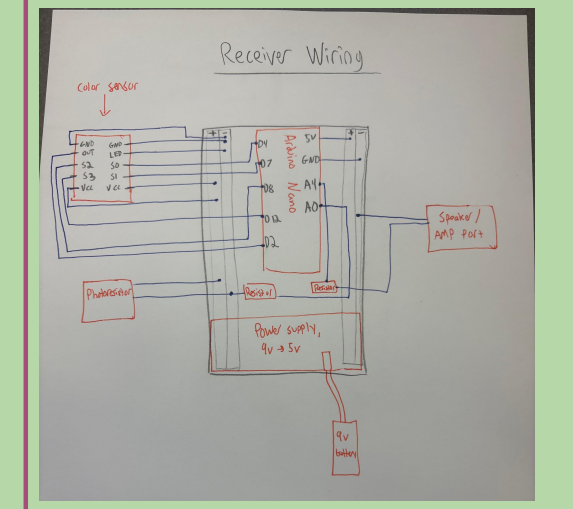
Sender Design

Receiver Design

Final Prototype

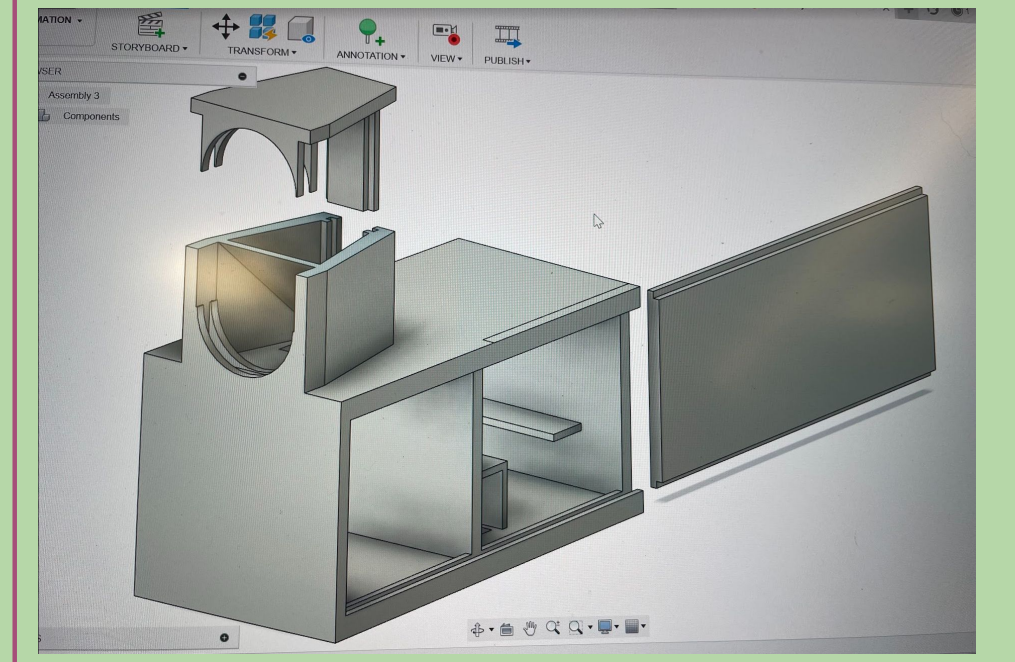


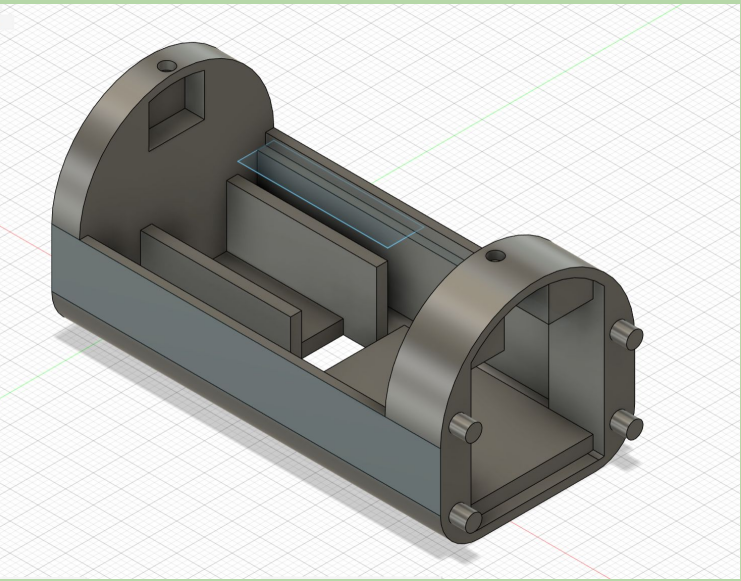




Description
- Sketches for receiver part
- Wiring for both parts

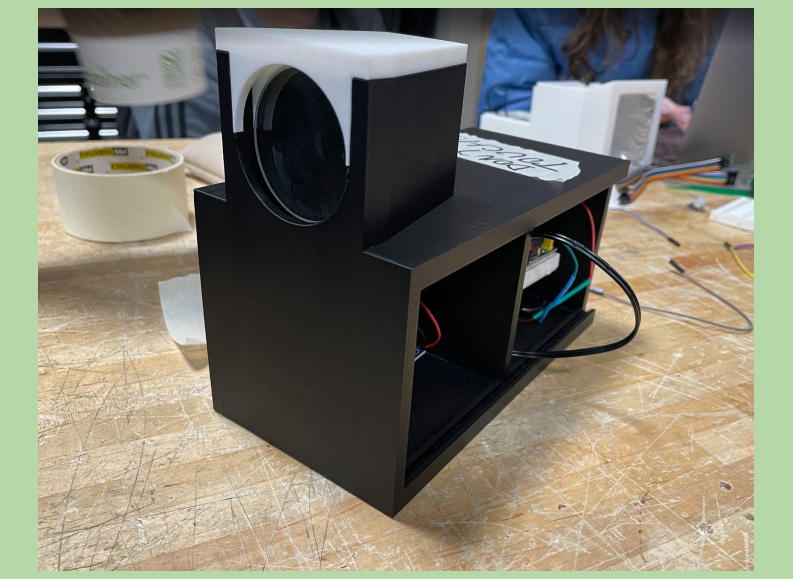
Learned
- Thought we wanted two circuits for the receiver
- All the dimensions for the components

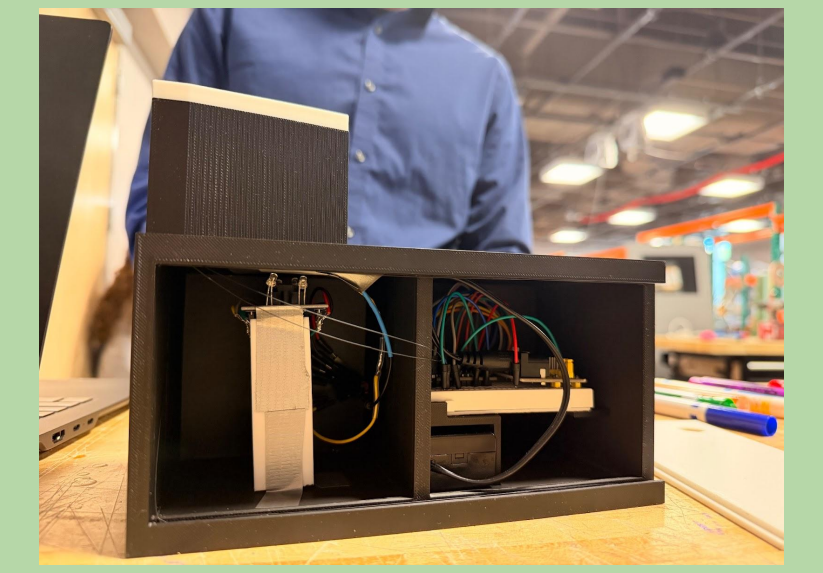


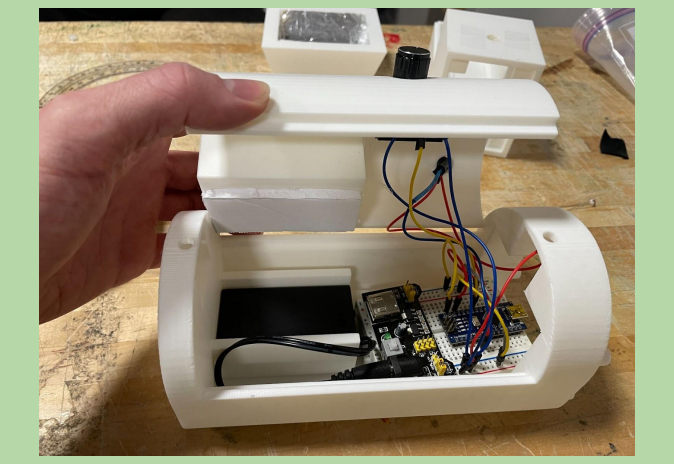


Description
- CAD designs for each piece
- Receiver has 3 pieces
- Sender has 4 pieces

Learned
- Tolerance is very important for physical designs
- Need to print multiple supporting pieces for installation purposes
- Small changes can impact printing time drastically







Description
- Final construction
- Used a focusing double convex lens for receiver, as well as a mirror to direct light to isolation chamber
- Installed piece to raise sensor to the top for better reading
- Installed a photoresistor to detect the amount of light being detected
- Installed a piece of paper for the color sensor to have something to read more clearly
- Sender now more ergonomic and practical

Final takeaways
- Sender worked exactly as intended
- Receiver suffers from consistency issues, but more or less functions properly
- More research and testing into the color sensor itself would likely have yielded better design solutions